

Dart SDK Integration Guide — SBQR Mobile Banking

Sep 26, 2026 · @Arif Ahmed

Step-by-step instructions for the Dart SDK team to wire real API calls the same way `rv1-sbqr-app-emulator` does, against the **dev** environment.

1. Purpose & scope

The FI's real mobile app must call the same two backend services the `rv1-sbqr-app-emulator` reference app calls, in the same order, with the same payload shapes. This guide is that contract, written for the **Dart SDK team** to implement an `sbqr_sdk` package the FI app depends on, targeting the shared **dev** environment (no local stack needed).

Out of scope: QR image rendering, camera/scan UI, and on-device ledger/UI screens — those are FI-app concerns. In scope: authentication, token lifecycle, and the three QR calls (generate static, generate dynamic, validate).

2. Architecture — who the app talks to

The app **never** talks to `sbqr.api` (the platform) directly, and never holds a platform `client_secret`. There are exactly two hosts the SDK calls, chained like this:

Hop	Caller	Callee	Credential used
1	Mobile app	FI Identity Provider (IdP)	Customer's own username/password
2	Mobile app	FI Gateway (BFF)	The customer JWT from hop 1, as a Bearer token
3 (server-side only, not the app's concern)	FI Gateway	<code>sbqr.api</code> (platform)	Gateway's own <code>client_id / client_secret</code> , swapped in server-side

Why this shape: the IdP and the Gateway have no CORS policy — they're built for native/programmatic callers, not browsers — so the emulator proxies hop 1/2 server-side too, but a native Dart app calls both hosts directly over HTTPS (no proxy needed off-

browser).

The Gateway (BFF) is a thin, verbatim proxy in front of `sbqr.api`'s public contract (`rv1-secure-bqr-manager/contracts/v1.public.json`): it validates the inbound customer JWT, attaches its own platform token, forwards the request unmodified, and returns `sbqr.api`'s response unmodified. This means the request/response JSON shapes documented in Steps 3–5 below are byte-for-byte what the Dart SDK sends and receives — there is no separate Gateway-specific schema to learn.

3. Dev environment reference

Service	Base URL	Role
FI IdP (Dhaka Bank mock)	<code>https://fi-idp-dhakabank.fly.dev</code>	Login, token refresh
FI Gateway (BFF)	<code>https://rv1-sbqr-fi-gateway.fly.dev</code>	QR generate/validate
<code>sbqr.api</code> (platform)	<code>https://sbqr-api-dev.fly.dev</code>	Not called by the app — reference only

Test customer (seeded in the IdP; password pattern is always `<username>@1234`):

Field	Value
Username	<code>fatema</code>
Password	<code>fatema@1234</code>
Customer name	Fatema Akter
Recipient PAN (account) used in examples	<code>1234567890123456</code>
Institution code	<code>000085</code> (Dhaka Bank)

Other seeded usernames for testing edge cases: `rafiq`, `karim` (normal accounts), `nasrin` (**locked account** — use to test the `account_locked` error path).

Health checks (no auth, useful for an SDK connectivity smoke test):

```
GET https://fi-idp-dhakabank.fly.dev/health
GET https://rvl-sbqr-fi-gateway.fly.dev/health/live
GET https://rvl-sbqr-fi-gateway.fly.dev/health/ready
```

health/ready on the Gateway must report **Healthy** (its platform token is pre-warmed) before Step 3–5 calls will succeed; if it 503s, wait a few seconds and retry.

4. Step 1 — Customer login

Purpose: authenticate the account holder against the FI's own IdP using the OAuth2 "Resource Owner Password Credentials" grant. The IdP checks the credential and account status (locked / expired password) and issues a signed access token plus a rotating refresh token. This is the exact grant a native banking app uses — no custom login endpoint.

```
POST https://fi-idp-dhakabank.fly.dev/connect/token
Content-Type: application/x-www-form-urlencoded
```

Request body (form-encoded, NOT JSON):

```
grant_type=password&username=fatema&password=fatema@1234
```

Response — **200 OK**:

```
{
  "access_token":
  "eyJhbGciOiJSUzI1NiIsImtpZCI6ImRldi1rZXktMSIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "openid profile sbqr.api",
  "id_token":
  "eyJhbGciOiJSUzI1NiIsImtpZCI6ImRldi1rZXktMSIsInR5cCI6IkpXVCJ9...",
  "refresh_token":
  "b1ab7ed269a180cc1952aec873f74ad27772e8ac7e00512398de5761dafb651b",
  "refresh_expires_in": 86400
}
```

Decoded **access_token** payload (RS256, verifiable against **GET /.well-known/jwks.json**):

```
{
  "name": "Fatema Akter",
  "preferred_username": "fatema",
  "customer_id": "CIF-00458821",
  "fi_id": "dhakabank",
  "iss": "https://fi-idp-dhakabank.fly.dev",
  "aud": "sbqr-fi-gateway",
  "sub": "user-fatema",
  "iat": 1790367284,
  "exp": 1790367584
}
```

Error responses — 400 Bad Request (OAuth2 shape, RFC 6749 §5.2):

```
{"error": "invalid_grant", "error_description": "Incorrect username or password."}
```

```
{"error": "account_locked", "error_description": "This account is locked. Contact your branch."}
```

SDK rule: `access_token` is the *customer* JWT — store it (and the refresh token) client-side; use it as the Bearer token for every Gateway call in Steps 3–5. Never send it to `sbqr.api` directly.

5. Step 2 — Token refresh

Purpose: the access token expires in 300s. Rather than re-prompting for a password, exchange the refresh token for a fresh pair. The refresh token **rotates on every use** — the old one is invalidated the moment a new one is issued, so the SDK must persist the new pair immediately and serialize concurrent refresh attempts (two simultaneous 401s must not both call refresh with the same stale token).

```
POST https://fi-idp-dhakabank.fly.dev/connect/token
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=refresh_token&refresh_token=b1ab7ed269a180cc1952aec873f74ad27772e8ac7e00512398de5761dafb651b
```

Response — 200 OK : identical shape to Step 1's response (new `access_token` + new `refresh_token`).

SDK rule (client-side session policy, matches the emulator):

1. Before any Gateway call, check the cached token's `exp` claim; treat it as expired if fewer than 20 seconds remain (avoids racing the wire).
2. If expired, call refresh; if a refresh is already in flight, await that same in-flight call rather than starting a second one.
3. If the IdP rejects the refresh token (400, e.g. `invalid_grant`), the session is over — clear stored tokens and force the user back to login. Do **not** treat an unreachable IdP the same way — keep the session and let the caller show a connectivity error instead.

6. Step 3 — Generate a static QR

Purpose: produce a reusable BanglaQR (send-money) code for the logged-in customer's own account, signed server-side by `sbqr.api`. "Static" = no amount encoded; the payer types the amount when they scan it.

```
POST https://rv1-sbqr-fi-gateway.fly.dev/v1/qr/generate/static
Authorization: Bearer <customer access_token from Step 1/2>
Content-Type: application/json
X-Correlation-Id: <client-generated UUID, one per request, for tracing>
```

Request body:

```
{
  "recipientName": "Fatema Akter",
  "recipientCity": "Dhaka",
  "recipientPan": "1234567890123456"
}
```

Optional fields (omit entirely if empty — don't send `null` or `""`): `postalCode`, `customerLabel`, `purposeOfTransaction`.

Field limits: `recipientName` ≤ 25 chars, `recipientCity` ≤ 15 chars, `recipientPan` ≤ 19 chars, `postalCode` ≤ 10 chars, `customerLabel` / `purposeOfTransaction` ≤ 25 chars.

Response — `201 Created`:

```
{
  "qrPayload":
    "00020101021126520014bd.org.bb.npsb010200020400850316123456789012345652044829
    53030505802BD5912Fatema
    Akter6005Dhaka80660014bd.org.bb.npsb0144/WSCPeFbeP6QwBGI9pHMWS8Tj3o55LPGwiJxn
    HsfBd8W81660014bd.org.bb.npsb0144LawqLKx6LFxdn702tWvQyKzp0MGADITjNcqbgGqyAw==
    630467C3",
  "payloadHash":
    "3af4e6a52fb81cbf38c7aed5416a1d62eedb4f43d96026df8cd793ee986d4af8",
  "qrType": "STATIC",
  "signatureKeyVersion": 1
}
```

`qrPayload` is the raw EMVCo TLV string — hand it to a QR-image renderer as-is; don't parse or re-encode it client-side.

Error responses (all `application/problem+json`, i.e. `ProblemDetails`):

Status	Meaning	SDK handling
400	Malformed/missing required field	Surface <code>detail / errors</code> to the user; this is a client bug, not retryable
401	Customer token expired/invalid	Silently refresh (Step 2) and retry once ; if the retry also 401s, force re-login
403	Not allowed	Surface as a hard stop, no retry
422	Field failed business validation	Surface <code>detail</code> to the user
409	Idempotency conflict (see below)	Surface a generic "try again" — do not silently retry with a new key

Idempotency: the Gateway accepts an optional `Idempotency-Key` header; if omitted, it derives one server-side from the user + request body within a rolling window, so an accidental double-tap on "Generate" during a slow network is safe and returns the same QR rather than issuing two.

7. Step 4 — Generate a dynamic QR

Purpose: same as Step 3, but the amount is fixed and encoded into the signed payload — used for "request money" flows. Recommended max lifetime for a dynamic QR is 15 minutes; the SDK should expire/regenerate it client-side after that even though the server doesn't enforce a TTL on the payload itself.

```
POST https://rv1-sbqr-fi-gateway.fly.dev/v1/qr/generate/dynamic
Authorization: Bearer <customer access_token>
Content-Type: application/json
X-Correlation-Id: <client-generated UUID>
```

Request body — identical to static, plus `transactionAmount` (required, string, digits with at most one decimal point, e.g. `"500.00"`, ≤ 13 characters):

```
{
  "recipientName": "Fatema Akter",
  "recipientCity": "Dhaka",
  "recipientPan": "1234567890123456",
  "transactionAmount": "500.00"
}
```

Response — `201 Created` (same shape as Step 3, `qrType` is now `DYNAMIC`):

```
{
  "qrPayload":
    "00020101021226520014bd.org.bb.npsb010200020400850316123456789012345652044829
    53030505406500.005802BD5912Fatema
    Akter6005Dhaka80660014bd.org.bb.npsb0144/WSCPeFbeP6QwBGI9pHMWS8Tj3o55LPGwiJxn
    HsfBd8W81660014bd.org.bb.npsb0144LawqLKx6LFxdn702tWvQyKzpOMGADITjNcqbGqyAw==
    6304D841",
  "payloadHash":
    "7dcb0dc9004b32c00e8b3b61135bf8e16747c7b6c778660a77c9228925477cf7",
  "qrType": "DYNAMIC",
  "signatureKeyVersion": 1
}
```

Same error table and idempotency behavior as Step 3.

8. Step 5 — Validate (scan) a QR

Purpose: when the app scans (or a user uploads an image of) someone else's QR, this call cryptographically verifies the issuing institution's Ed25519 signature against the Bangladesh Bank trust store before the app lets the user proceed to pay. **Never initiate a transfer without a `VALID` verdict here.**

```
POST https://rv1-sbqr-fi-gateway.fly.dev/v1/qr/validate
Authorization: Bearer <customer access_token>
Content-Type: application/json
X-Correlation-Id: <client-generated UUID>
```

Request body — `requestId` and `requestTimestamp` must be fresh on every attempt (they feed the platform's replay-window check; reusing them on a retry will draw `REQUEST_REPLAYED`):

```
{
  "qrPayload": "<the decoded QR string as scanned>",
  "requestId": "a1b2c3d4e5f6...",
  "requestTimestamp": "2026-09-26T10:15:30.000Z"
}
```

Response — `200 OK`:

```
{
  "verdict": "VALID",
  "trustSource": "TRUST_DIRECTORY",
  "reasonCode": null,
  "institutionCode": "000085",
  "payloadHash":
    "7dcb0dc9004b32c00e8b3b61135bf8e16747c7b6c778660a77c9228925477cf7",
  "recipientName": "Fatema Akter",
  "recipientPan": "1234567890123456",
  "qrClassification": "P2P"
}
```

verdict values and required app copy (never show the raw enum to the user):

Verdict	Meaning	User-facing message
<code>VALID</code>	Safe to proceed	(proceed to review/pay screen)

Verdict	Meaning	User-facing message
<code>INVALID_SIGNATURE</code>	Signature check failed	"This QR could not be verified. Please ask the recipient to share a new one."
<code>STRUCTURAL_INVALID</code>	Malformed TLV	"This QR appears to be damaged. Please try again or request a fresh QR."
<code>KEY_NOT_FOUND</code>	Institution not in BB trust store	"The recipient's institution is not in the Bangladesh Bank trust store."
<code>KEY_SUSPENDED</code> / <code>KEY_REVOKED</code> / <code>KEY_NOT_ACTIVE</code>	Institution key not currently trusted	"The recipient institution's signing key is not currently trusted. The payment was blocked for your safety."
<code>NON_P2P</code>	It's a merchant (P2M) QR	"This is a merchant QR, not a personal QR. Please use merchant payment instead."
<code>REQUEST_STALE</code>	Device clock drift	"Please correct your device date and time, then try again."
<code>REQUEST_REPLAYED</code>	Duplicate <code>requestId</code>	"This verification was already processed. Please scan the QR again."

Every non- `VALID` verdict is a **block**, not a warning — the SDK must not expose a way to bypass it and pay anyway.

9. Error handling — two distinct shapes

The IdP and the Gateway return **different error envelopes** — the SDK needs one parser per host, not one shared one:

IdP errors (OAuth2, RFC 6749 §5.2) — always `400` :

```
{ "error": "invalid_grant", "error_description": "Incorrect username or password." }
```

Switch on `error` (`invalid_grant`, `account_locked`, etc.); `error_description` is safe to show the user as-is.

Gateway/platform errors (RFC 7807 Problem Details) — `400 / 401 / 403 / 409 / 422`:

```
{ "type": null, "title": "...", "status": 422, "detail": "...", "instance": null }
```

Prefer `detail`, fall back to `title`, for user-facing text.

Connectivity vs. rejection: a `0 / 502 / 503 / 504` (or a network exception) means the backend itself is unreachable — show “can’t reach the server, check your connection,” and it is safe to let the user retry manually. A `401 / 403 / 4xx` is the backend actively rejecting the request — do not treat it as a connectivity issue.

401 retry policy: exactly one silent retry per call, after a forced token refresh. If the retried call also 401s, stop — log the user out and surface “your session has expired.” Never loop on 401.

Correlation IDs: always send a fresh `X-Correlation-Id` per request to the Gateway; log it alongside any error you report, so the FI’s ops team can grep it out of Gateway logs when the SDK team needs help debugging a specific failed call.

10. Recommended Dart SDK layout

Mirror the emulator’s own `src/api` split (it’s already proven against these exact endpoints) rather than inventing a new structure:

File	Responsibility
<code>http_client.dart</code>	One low-level POST helper: sets headers, generates <code>X-Correlation-Id</code> , parses JSON, maps non-2xx to a typed <code>ApiException</code> (carrying status + parsed body)
<code>auth_api.dart</code>	<code>signIn(username, password)</code> , <code>refresh(refreshToken)</code> , JWT decode helper, typed <code>AuthException</code> for OAuth2 <code>error</code> codes

File	Responsibility
<code>qr_api.dart</code>	<code>generateStatic()</code> , <code>generateDynamic()</code> , <code>validateQr()</code> — each wrapped in a <code>withAuth</code> helper that supplies the bearer token and retries once on 401
<code>session.dart</code>	Holds the current token pair, exposes a <code>tokenSource</code> callback the QR API uses, owns the refresh-in-flight de-duplication and the 20-second-early-expiry check

Keep IdP and Gateway base URLs as build-time config (e.g. `--dart-define=IDP_URL=...` / `--dart-define=BFF_URL=...`), never hard-coded, so QA/stage/prod can be swapped without a code change — same principle as the emulator's `.env`.

11. Security rules — do's and don'ts

- **Do** send the customer's own JWT (from Step 1/2) to the Gateway as `Authorization: Bearer`. **Don't** ever request, store, or embed a platform `client_id` / `client_secret` in the app — that credential belongs only to the Gateway, server-side.
- **Do** treat `refresh_token` as single-use and store the newest one only; discard the old one the instant a refresh succeeds.
- **Do** store both tokens in secure, OS-backed storage (Keychain / Keystore via `flutter_secure_storage` or equivalent) — never plain `SharedPreferences`, never logged.
- **Don't** log request/response bodies containing `password`, `access_token`, or `refresh_token` in cleartext, even in debug builds — mask them (the emulator's own dev panel masks `password` and `refresh_token` for this reason).
- **Do** validate every scanned QR via Step 5 before showing a payment review screen, and block on any non-`VALID` verdict — no client-side override.
- **Do** call all three hosts over HTTPS only; pin nothing extra is required in dev, but plan for certificate pinning before a prod build.
- **Don't** cache or hard-code the `sbqr-api-dev.fly.dev` host in the SDK — the app has no legitimate reason to reach it; if a build ever needs to, that's a sign the Gateway contract is being bypassed.